

MOCK STOCK EXCHANGE

Master Product Plan, Architecture, Mathematical Model & Build Instructions

Purpose: Build a live, multiplayer mock stock exchange for approximately 40–50 human participants, supported by algorithmic trading agents and pre-loaded news events.

Core philosophy: The exchange should not arbitrarily set prices. Human and AI traders submit orders; the order book and matching engine discover the traded price.

1. Executive Concept

The project is a miniature stock exchange that participants can trade through a proper terminal. Each participant receives virtual capital and can buy and sell fictional stocks. The exchange maintains order books, matches compatible orders, records trades, updates portfolios and publishes live market data. AI traders provide additional liquidity and competition. Pre-loaded news events create controlled market shocks and change the behaviour or expectations of the agents.

The key distinction

The mathematical model influences expectations and trader behaviour; the order book determines actual execution prices. This prevents the simulation from feeling like a game in which an organizer simply says that a stock is +5%.

2. Final System Architecture

Layer	Responsibility
Human traders	Place market/limit orders through trading terminals.
AI traders	Use different strategies to decide price, side and quantity.
News engine	Releases pre-loaded, manually rated events at scheduled times.
Strategy engine	Converts news, price, fundamentals and portfolio state into agent decisions.
Order gateway	Validates and accepts orders.
Order books	Maintain bids and asks for every stock.
Matching engine	Matches compatible buy and sell orders.
Trade/settlement engine	Transfers virtual cash and shares and records executions.
Market data	Publishes last price, bid, ask, volume, OHLC and depth.
Portfolio/P&L	Calculates holdings, cash, equity, realized/unrealized P&L.
Leaderboard	Ranks participants using return and risk metrics.
Admin/NSE dashboard	Controls sessions, news, halts, stocks and market state.

3. Development Layers

Layer 0 — Foundation — Backend project, database, users, stocks, configuration, tests.

Layer 1 — Core entities — Trader accounts, cash, holdings, stock definitions, portfolio valuation.

Layer 2 — Orders — Market orders, limit orders, cancellation, validation and order status.

Layer 3 — Order book — Bid/ask queues, price-time priority, best bid/ask, depth.

Layer 4 — Matching engine — Actual trade execution, partial fills, settlement and last traded price.

Layer 5 — Trading terminal — Watchlist, chart, order book, buy/sell ticket, positions, orders, trades and P&L;

Layer 6 — Real-time market — WebSockets, live market data, synchronized terminals.

Layer 7 — AI traders — Market maker, momentum, value, mean reversion, FOMO, panic and noise agents.

Layer 8 — News — Pre-loaded events with direction, impact, confidence, scope and decay.

Layer 9 — Market model — Fair value, volatility, sentiment/agent pressure, liquidity and news effects.

Layer 10 — Competition — Leaderboard, risk metrics, rankings and event scoring.

Layer 11 — Admin/NSE — Market controls, news release, halts, circuit breakers, scenario controls.

4. The Order Book — The Core of Price Discovery

Every stock has a buy side (bids) and sell side (asks). A limit order specifies a maximum buy price or minimum sell price. A market order accepts the best available prices. Orders are prioritized primarily by price and then by time.

Example starting book:

Ask: ■101.00 × 100, ■102.00 × 150, ■103.00 × 100

Bid: ■99.00 × 100, ■98.00 × 200, ■97.00 × 300

If a market buyer wants 150 shares, the exchange consumes 100 at ■101 and 50 at ■102. The last executed trade becomes ■102. The buyer moved the price by consuming liquidity.

Important: An order that does not execute does not automatically change the last traded price. The market price is based on actual executions, while the order book shows the current supply/demand available for execution.

5. Matching Rules

- Buy limit order can match a sell if bid price $\geq$ ask price.
- Sell limit order can match a buy if ask price $\leq$ bid price.
- Best price has priority; at the same price, earlier order has priority.
- Partial fills are allowed.
- Unfilled limit quantity remains in the order book.
- Market orders consume the best available opposite-side liquidity.
- Every execution creates a trade record containing stock, quantity, price, buyer, seller and timestamp.
- Last traded price (LTP) is updated from actual executions.

6. Core Mathematical Model

The model is deliberately modular. We do not want one formula to directly force the stock price. Instead, the model creates signals that influence AI behaviour and, where appropriate, a slowly moving reference/fair value.

6.1 Individual sentiment

Each agent can have a sentiment/stance toward a stock between -1 and $+1$:

-1 = strongly bearish 0 = neutral $+1$ = strongly bullish

6.2 Weighted market sentiment

Different traders can have different influence based on capital, strategy or a configured weight:

$$S = \frac{\sum(\text{weight}_{\blacksquare} \times \text{sentiment}_{\blacksquare})}{\sum(\text{weight}_{\blacksquare})}$$

6.3 Order-flow pressure

Use actual executed or submitted notional flow to measure directional pressure. A simple normalized version is:

$$P = \frac{(\text{BuyNotional} - \text{SellNotional})}{(\text{BuyNotional} + \text{SellNotional})}$$

P approaches $+1$ when buying dominates and -1 when selling dominates. For a first prototype, this can be calculated over a rolling time window.

6.4 Combined market force

A configurable weighted signal can combine order pressure, sentiment, news and AI-agent pressure:

$$M = w_{\blacksquare}P + w_{\blacksquare}S + w_{\blacksquare}N + w_{\blacksquare}A$$

Where P = order pressure, S = weighted sentiment, N = normalized news impact, A = AI strategy pressure. The weights should sum to 1. A sensible starting configuration is $w_P=0.40$, $w_S=0.25$, $w_N=0.20$, $w_A=0.15$. These are tuning parameters, not financial laws.

6.5 Fair-value pull / mean reversion

$$\text{Correction} = \alpha(F - \text{Price})$$

F is the engine's hidden/reference fair value and α is deliberately small. This prevents the simulated price from drifting forever and allows fundamentals to matter without overriding the order book.

6.6 Optional reference-price movement

$$\text{ReferencePrice}_{t+1} = \text{ReferencePrice}_t \times (1 + kM)$$

k controls the sensitivity of the reference model. This should not replace the matching engine. It can instead influence AI quotes, fair value estimates or liquidity when the market is inactive.

6.7 Noise

$$\text{Signal}_t = M_t + \varepsilon_t$$

ε is a small bounded random term. Keep it controlled so randomness adds uncertainty without destroying the scenario.

7. News Engine

News will be pre-loaded and manually rated. This removes the need for a separate NLP sentiment engine and makes the event predictable and easy to manage.

Field	Meaning	Example
Event	Human-readable headline	TechNova wins government contract
Direction	-1 to +1	+1
Impact	Magnitude, usually 0 to 1	0.75
Confidence	How strongly agents should trust it	0.90
Scope	Company, sector or market	Tech sector
Duration	Approximate effect window	20 minutes
Decay	How impact fades over time	Exponential
Fundamental effect	Optional change to fair value	+8%

News impact decay

$$I(t) = I \cdot e^{(-\lambda t)}$$

I is initial impact, t is time since release and λ controls how quickly the event fades. For example, a short-lived rumor can have a high λ ; a structural policy change can have a low λ .

8. AI Trader Agents

Agent	Core behaviour	Typical reaction to bullish news
Market Maker	Posts two-sided quotes; manages inventory and spread.	Moves quotes upward while controlling inventory.
Momentum	Buys strength, sells weakness.	Aggressive buy if momentum confirms.
Mean Reversion	Trades deviations from fair/reference value.	May sell if price becomes excessively high.
Value Investor	Compares price with estimated fair value.	Raises fair value estimate and buys if price is below it.
FOMO Trader	Chases rapid price moves.	Buys aggressively after a sharp rise.
Panic Trader	Overreacts to negative signals.	Sells aggressively after bad news.
Noise Trader	Randomized decisions within risk limits.	Unpredictable small flow.

9. Stock Model

Start with 10–15 fictional companies rather than hundreds. Each stock should have a different personality.

Attribute	Purpose
Ticker / name	Identification.
Starting price	Initial market reference.
Shares outstanding	Market capitalization.
Hidden/reference fair value	Fundamental anchor.
Volatility class	Low, medium or high movement sensitivity.
Liquidity class	How much AI market-making support it receives.
Sector	Allows sector-wide news.
Fundamental profile	Growth, stable, distressed, etc.

10. Risk & Exchange Rules

- Virtual starting capital, e.g. ■10,00,000 per participant.
- Position limits to prevent one trader from controlling an entire stock.
- Optional maximum order size.
- Upper/lower circuit limits to prevent pathological moves.
- Trading halt mechanism for extreme volatility or admin events.
- Tick size, e.g. ■0.05 or ■0.10.
- No negative cash or impossible short positions unless explicitly supported.
- Every order and trade must be logged for auditability.
- Admin override should be possible, but manual price manipulation should not be the normal mechanism.

11. Participant Trading Terminal

Each participant should see a professional but simple terminal containing:

- Current price, daily change and volume.
- Candlestick/line chart.
- Live bid/ask order book.
- Buy/Sell order ticket.
- Market and limit order options.
- Open orders and cancellation.
- Positions and average entry price.
- Available cash and portfolio value.
- Realized and unrealized P&L;
- Trade history.
- Watchlist.
- Leaderboard.
- News feed.

12. Admin / 'NSE' Control Panel

- Start, pause and resume market.
- Open/close individual stocks.
- Release the next pre-loaded news event.
- Schedule news events.
- Trigger market-wide or sector events.
- Trigger trading halt.
- View all order books.
- View all trades and participants.
- Monitor AI inventory and behaviour.
- Configure bot capital, aggressiveness and position limits.
- Adjust scenario parameters between test runs.

13. Leaderboard & Scoring

Profit should matter, but a pure return ranking rewards reckless gambling. Track multiple metrics:

Metric	Purpose
Portfolio value	Absolute wealth at the end.
Return %	Performance relative to starting capital.
Realized P&L	Profit/loss from closed trades.
Unrealized P&L	Mark-to-market P&L on open positions.
Maximum drawdown	Worst peak-to-trough decline.
Sharpe ratio	Risk-adjusted return.
Win rate	Percentage of profitable closed trades.
Turnover	Trading activity.

14. Recommended Technology Stack

Component	Technology	Reason
IDE / vibe coding	Cursor or VS Code + AI	Fast iterative development.
Frontend	Next.js + TypeScript	Interactive trading terminal.
Backend	Python + FastAPI	Good for exchange APIs and quantitative logic.
Database	PostgreSQL	Reliable relational storage.
ORM	SQLAlchemy	Database abstraction.
Realtime	WebSockets	Live price/order-book updates.
Version control	Git + GitHub	Safe iteration and rollback.

15. Project Structure

```
mock-stock-exchange/  
  ■■■■ backend/  
    ■ ■■■■ app/  
      ■ ■ ■■■■ models/
```

```
■ ■ ■■ schemas/
■ ■ ■■ api/
■ ■ ■■ services/
■ ■ ■■ exchange/
■ ■ ■■ main.py
■ ■■ tests/
■ ■■ requirements.txt
■■■ frontend/
■■■ docs/
■■■ README.md
■■■ .gitignore
```

16. Build Order & Acceptance Tests

Layer 0: Project starts; database connects; basic API health check works.

Layer 1: Can create two traders and one stock; cash and holdings are tracked.

Layer 2: Can submit, validate, cancel and inspect market/limit orders.

Layer 3: Order book shows correct bids/asks and price-time priority.

Layer 4: Two traders can execute a trade; partial fills and settlement work.

Layer 5: A participant can trade from a browser terminal.

Layer 6: Two browser sessions see the same market update in real time.

Layer 7: Three AI agents trade without breaking accounting or exchange rules.

Layer 8: A scheduled news event changes agent decisions.

Layer 9: Reference/fair-value and volatility models influence agents without directly overriding executions.

Layer 10: Leaderboard and risk metrics calculate correctly.

Layer 11: Admin can run an entire scenario from one dashboard.

17. First Prototype — The Smallest Possible Market

Before building the full 40–50 person event, prove the exchange with one stock and two simulated traders:

TECHNOVA — starting price ■100

Trader A — ■10,00,000 cash

Trader B — ■10,00,000 cash

A submits BUY 100 @ ■100. B submits SELL 100 @ ■100. The matching engine executes 100 shares at ■100. A receives 100 shares and pays ■10,000. B receives ■10,000 and gives up 100 shares. Last traded price becomes ■100.

Only after this works should the project move forward. The core exchange must remain correct before AI, news or advanced mathematics is added.

18. Vibe-Coding Rules

- Never ask the coding AI to build the entire project in one prompt.
- Give it one layer at a time.
- Ask it to explain architecture before large changes.
- Keep matching/settlement independent from AI and news.

- Write tests for every exchange rule.
- Commit working versions to Git frequently.
- Do not let AI directly modify the last traded price just to create movement.
- Prefer configuration values over hard-coded behaviour for bot aggressiveness, weights and news impact.
- When something breaks, fix the smallest layer responsible rather than rewriting the project.

19. Example Event Scenario

A 2-hour competition could contain 10–15 pre-loaded events. Example sequence:

Time	Event	Impact	Expected agent behaviour
10:05	TechNova beats earnings	+0.70	Momentum/FOMO buy; value investor raises fair value.
10:15	Banking regulation	−0.50	Banking-focused agents reduce exposure.
10:25	Oil price shock	−0.40	Energy-sensitive stocks react; market maker widens spread.
10:40	Government subsidy	+0.80	Sector momentum and FOMO buying.
10:55	CEO resignation	−0.90	Panic selling and short-term liquidity stress.
11:10	Rate cut	+0.60	Broad risk-on response.
11:25	Fraud allegation	−1.00	Heavy selling; possible trading halt.

20. What We Are NOT Building Initially

- Real-money trading.
- Connection to NSE or any live exchange.
- Machine-learning sentiment analysis.
- High-frequency trading infrastructure.
- Hundreds of stocks.
- Complex derivatives.
- Real-world financial advice or prediction.
- A formula that directly dictates every stock price.

21. Final Design Principle

Humans and bots create orders. The exchange matches orders. Executed trades create the observed price. News and mathematical models influence trader behaviour and reference values. The admin controls the scenario, not the outcome.

This gives us a controlled simulation that is realistic enough to feel like a market, but configurable enough to run as a college competition.